浙江大学

本科实验报告

课程名称：信息控制与计算：远程声控系统

姓　　名：

学　　院：信息与电子工程学院

系：

专　　业：信息工程

学　　号：

指导教师：陈晓明

2025 年 12 月 25 日

目录

一、 摘要 4

二、 问题分解 4

2.1 信号获取与数字化 4

2.2 噪声信道下的可靠传输挑战 4

2.3 语义特征提取与识别挑战 4

2.4 受限条件下的精准控制挑战 5

三、 系统总体架构 5

1. 通信系统： 5

2. 计算系统： 5

3. 控制系统： 5

四、 系统 I:信号处理与通信子系统 6

1. 信号采集与预处理： 6

2. 信源编码：PCM 与 Huffman 9

3. 信道编码与帧同步 11

4. BPSK 调制与相干解调 13

5. 帧同步与信道解码 15

6. 信源解码 17

7. 子系统总结 19

五、 系统 II:计算语音识别子系统 20

1. 特征工程 20

2. KNN 分类器 21

3． 子系统总结 21
六、 系统 III:现代控制系统设计 22
1． 物理对象建模 22
2． 状态观测器设计 22
3． 控制律设计 23
4． 仿真循环 23
5． 子系统总结 25
七、 总结与心得 26
八、 参考文献 26

专业：信息工程
姓名：
学号：
日期：2025.12.25
地点：玉泉

浙江大学实验报告

课程名称：信息控制与计算
指导老师：陈晓明
成绩：
实验名称：远程声控系统

一、摘要

本报告详细阐述了一个集语音信号采集、数字通信传输、模式识别与现代控制于一体的远程声控系统。

系统首先通过 PCM 量化与 Huffman 编码实现信源压缩，利用 Hamming(7,4) 码与 Barker 序列实现信道纠错与帧同步；随后在 AWGN 信道下采用 BPSK 调制传输；接收端解调并恢复信号后，提取时频特征并通过 KNN 分类器识别指令；最终将指令映射为参考输入，驱动一个基于状态观测器与极点配置的质量-弹簧-阻尼系统，实现了在部分状态不可测情况下的闭环控制。

二、问题分解

2.1 信号获取与数字化

Q：如何将模拟语音信号转换为计算机可处理的数字信号，同时去除环境噪声干扰？

A：1.采样与量化：依据奈奎斯特采样定理选择合适的采样率，并利用 PCM 技术实现模拟信号的数字化。2.信号预处理：原始录音中往往包含直流偏置和静音片段，需要设计算法去除直流分量并进行端点检测，提取有效语音段。

2.2 噪声信道下的可靠传输挑战

Q：如何在有限带宽和噪声干扰的双重约束下，保证指令信息的完整性和准确性？

A：1.信源压缩：针对语音信号数据存在冗余的特性，需设计高效的编码方案以降低传输比特率。2.差错控制：为了对抗信道噪声，需引入信道编码技术，通过增加冗余位来实现自动纠错。3.帧同步：在连续的比特流中，接收端无法自动区分数据包的边界。需设计特殊的帧头序列以实现精准的帧同步。4.调制解调：需选择合适的数字调制方式将比特流搬移至高频载波，以适应信道传输特性。

2.3 语义特征提取与识别挑战

Q：如何建立从“物理波形”到“逻辑指令”的映射关系？

A：1.特征工程：原始时域波形维度过高，需提取具有强区分度的物理特征。本系统需分析不同指令在时域和频域的差异。2.模式分类：基于提取的特征向量，设计分类算法（KNN），计算待测样本与参考样本的距离，实现对用户意图的判决。

2.4 受限条件下的精准控制挑战

Q：如何在部分状态不可测的情况下，驱动系统实现快速、无静差的轨迹跟踪？

A：1.状态重构：针对速度不可测的问题，需基于现代控制理论设计状态观测器，利用输出误差重构系统全维状态。2.闭环反馈：设计状态反馈控制器，通过极点配置优化系统的动态响应速度和稳定性。3.指令跟踪：为了消除稳态误差并实现对正弦波等复杂轨迹的跟踪，需引入前馈控制环节。

三、系统总体架构

本系统模拟了真实场景下的远程控制流程，采用了模块化设计思想，数据流从声源端出发，经由有损信道传输，最终作用于被控对象。系统逻辑架构分为以下三大核心子系统：

1．通信系统：

通信子系统负责将模拟语音信号转换为抗干扰能力强的数字比特流，并确保其在加性高斯白噪声（AWGN）信道中的可靠传输。主要包括：语音采集、预处理、信源编码、信道编码；调制、模拟加性高斯白噪声 (AWGN) 信道及信号衰减；解调、解码

2．计算系统：

计算子系统充当系统的“大脑”，负责对解调后的语音信号进行分析，识别用户的意图。该部分包含特征工程与分类决策两个模块

3．控制系统：

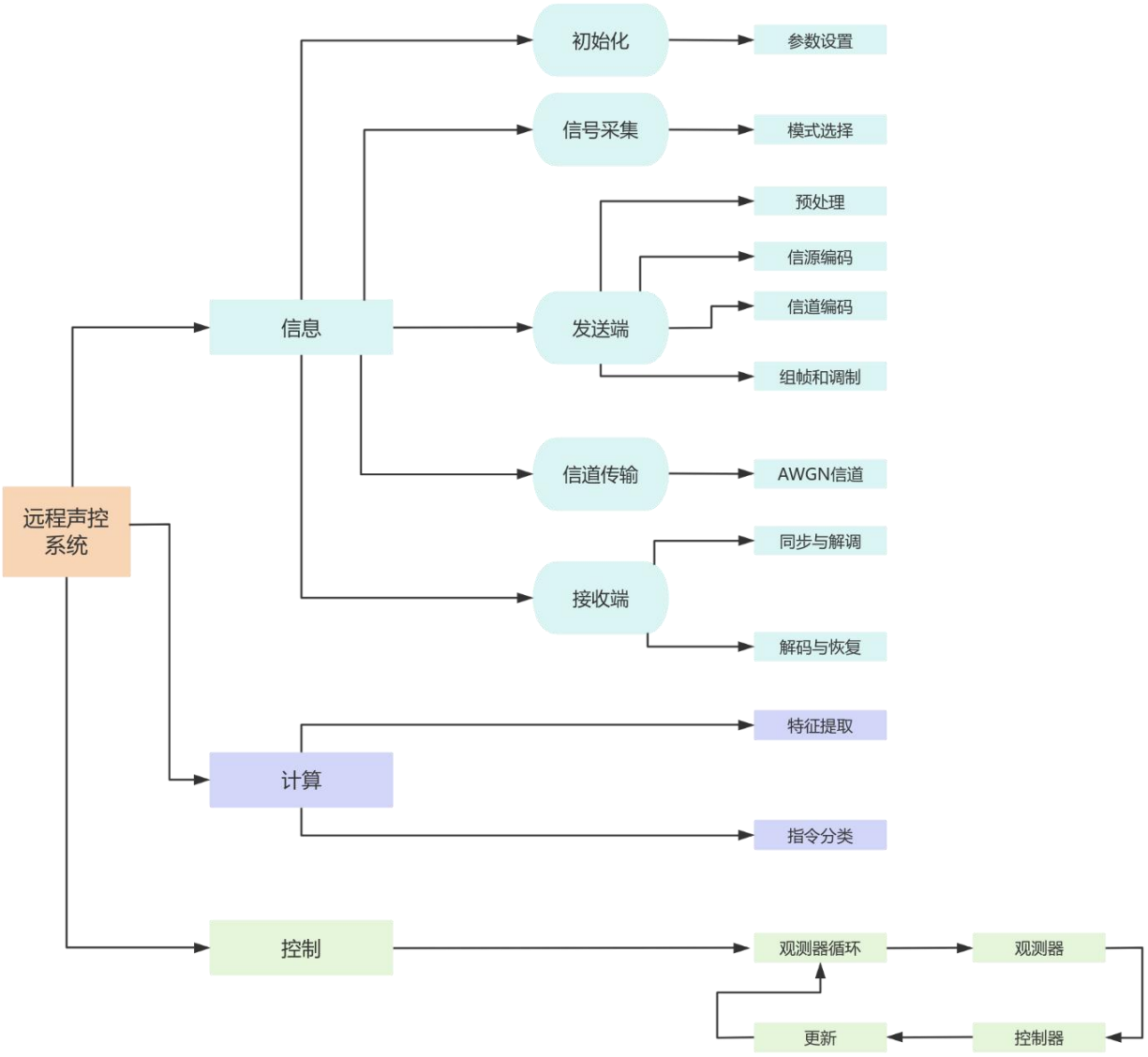
控制子系统负责执行识别出的指令，驱动一个二阶物理系统（质量-弹簧-阻尼系统）。为了解决传感器受限问题，系统引入了现代控制理论中的观测器设计

系统的伪代码如下：

Algorithm：远程声控系统			
	模块	核心功能	处理逻辑
信息	输入	信号获取	判断是实时录音还是读取仿真文件，并进行能量归一化
	编码	压缩纠错	PCM 量化——>Huffman 信源压缩——>Hamming 编码

	传输	调制同步	添加 13 位 Barker 码作帧头，使用 BPSK 调制通过噪声信道
	解码	恢复信号	利用互相关检测帧头，纠正传输误码，逆变换恢复语音波形
控制	识别	指令判定	提取过零率与谱质心等特征，通过 KNN 识别指令意图
计算	控制	状态反馈	基于 Luenberger 观测器估计不可测状态，实现闭环控制

系统的流程图如下：



四、系统 I:信号处理与通信子系统

1. 信号采集与预处理：

系统采样率设定为$f_s = 8000Hz$，本系统采用实时语音和仿真语音两个模式可供选择：

对于实时语音，由于人声说话的有效频率在 300Hz～3400Hz 之间，8000Hz 是记录人声的标准采样率，而每个样本点用 16bit 来存储，并设置为单通道，记时间为 1s

```
recObj = audiorecorder(fs, 16, 1);
recordblocking(recObj, 1.0);
raw_sig = getaudiodata(recObj);
current_title = '实时录音信号';
active_model = voiceKnnModel;
```

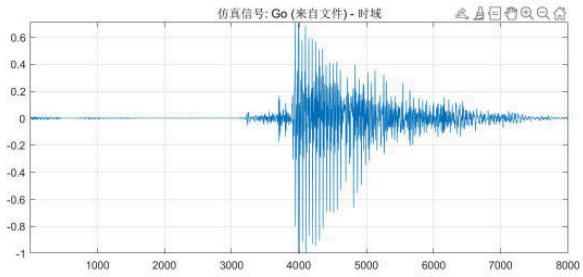
对于提前录制好的语音，直接读取文件后处理，由于使用手机录制因此需要先转化为单声道，然后重新采样为 8000Hz，由于录制过程中可能开头结尾有静音，因此保留峰值前后 0.5s，语音输入设为 1s，最后补零[1]

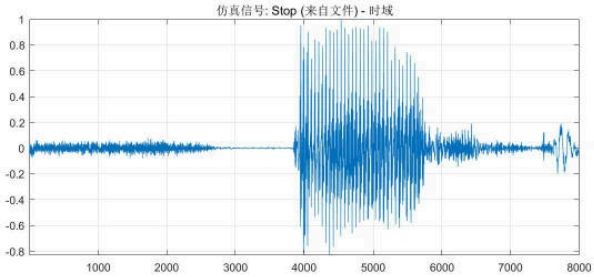
```
% 立体声转单声道
if size(file_sig, 2) > 1
file_sig = mean(file_sig, 2);
end
% 重采样到 8000Hz
if fs_file ~= fs
file_sig = resample(file_sig, fs, fs_file);
end
% 去掉录音开头和结尾可能存在的长静音
frame_len = 0.02 * fs;
energy = smooth(abs(file_sig), 100);
[~, max_idx] = max(energy);
half_len = floor(0.5 * fs);
start_idx = max(1, max_idx - half_len);
end_idx = min(length(file_sig), max_idx + half_len - 1);
raw_sig = file_sig(start_idx:end_idx);
% 补零
if length(raw_sig) < fs
padding = zeros(fs - length(raw_sig), 1);
raw_sig = [raw_sig; padding];
else
raw_sig = raw_sig(1:fs);
End
```

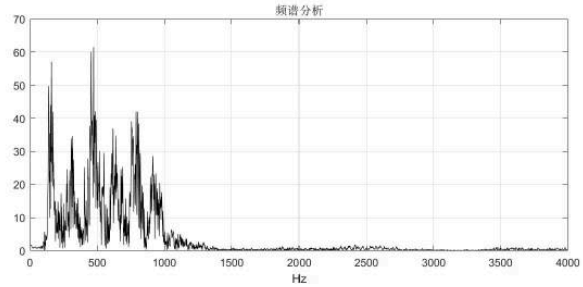
最后将得到的 signal 归一化到-1 到 1 的范围，便于后续处理

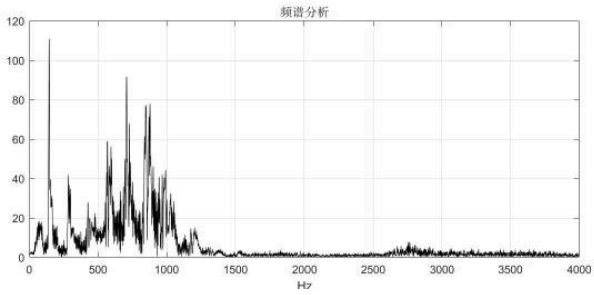
```
raw_sig = raw_sig / max(abs(raw_sig));
```

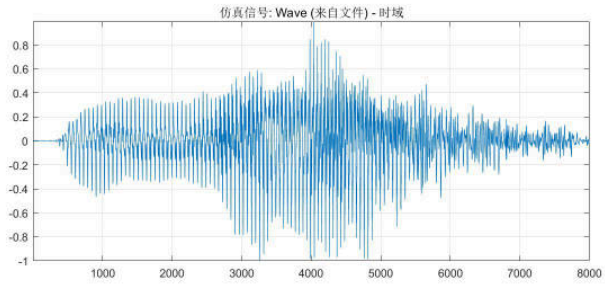
得到的三个语音信号 Go、Stop、Wave 的时域图和频域图如下：

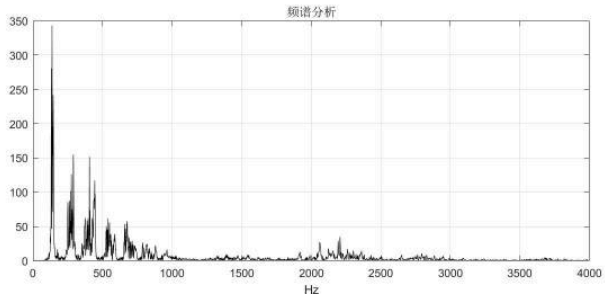












分析三个语音信号可以看出：

（1）Go：时域特征：突发性：信号呈现出典型的“短时爆发”形态。在大约采样点 3500 处有一个突然的能量爆发，然后迅速衰减；短促。频域特征：集中在低频，可以看到几个明显的峰值

（2）Stop：时域特征：看起来更“毛糙”或“密集”，是因为包含清辅音；频域特征：虽然低频（元音部分）依然有能量，但在 2000 Hz - 3500 Hz 的区间内，依然可以看到明显的能量分布，高频能量占比比较大

（3）Wave：时域特征：长持续时间，呈现出“渐强再渐弱”的特征，周期性强:，波形看起

来比较平滑；频域特征：极高纯度: 频谱非常干净，能量极度集中在极低频，过零率会非常低且稳定

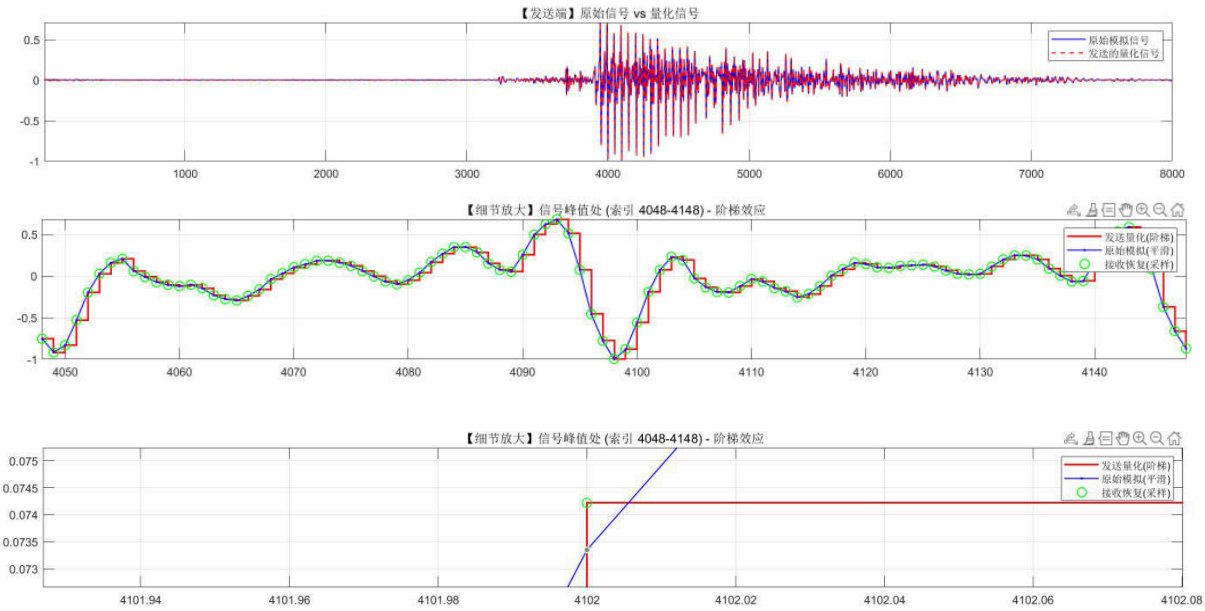
2．信源编码：PCM 与 Huffman

（1）PCM——脉冲编码调制：

PCM 是一种将时间连续、幅度连续的模拟信号，转换为时间离散、幅度离散的数字信号的调制方式。语音信号采样后进行量化，将连续的电压值变成离散值，此处采用的是均匀量化的方法，因此量化步长为$\Delta = \frac{x_{\max}-x_{\min}}{L}$，量化向下取整 $\text{index} = \text{floor}(\frac{x-x_{\min}}{\Delta})$，量化电平为$x_q = \Delta \cdot \left[\frac{x-x_{\min}}{\Delta}\right] + \frac{\Delta}{2} + x_{\min}$

```
min_val = -1; max_val = 1; L = 256;
delta = (max_val - min_val) / L;
q_indices = floor((signal - min_val) / delta);
q_indices(q_indices < 0) = 0;
q_indices(q_indices > L-1) = L-1;
quantized_sig = q_indices * delta + min_val + delta/2;
```

得到量化后的信号图像为：



量化噪声，PCM 过程引入了不可逆的误差，误差定义为：$e(n) = x(n) - Q(x(n))$，由于采用均匀量化，误差服从均匀分布，8bit 量化误差$\text{SQNR}_{dB} \approx 6.02 \cdot B + 1.76 \approx 50\,\text{dB}$，可以看出这个量化信噪比很高，带来的量化误差几乎可以忽略不计

（2）Huffman 编码——基于统计学的无损压缩：

Huffman 编码是一种熵编码，属于无损压缩算法。[2]它依据香农信源编码定理，利用符号出现的概率来构建最优的前缀码，语音信号可以压缩是因为数据存在冗余，信息的不确定性熵定义为$H(X) = -\sum_{i=1}^{n} P(x_i)\log_2 P(x_i)$，语音中，大概率出现的是“静音”或“低幅值信号”，小概率出现“爆破音”或“最大幅值”，这种概率分布是不均匀的

Huffman 算法通过构建二叉树来分配编码：统计概率，计算每个 PCM 量化值出现的频率；将概率最小的两个节点合并为一个新节点，其概率为两者之和；重复上述步骤，直到所有节点合并为一颗根树，左分支标 0，右分支标 1，从根到叶子的路径就是该符号的二进制码，所以对于高频符号较靠近树根，码字短；低频符号远离树根，码字长

```
huff_bits = huffmanenco(q_indices, dict);
```

这里我采用了两种方式构建 Huffman 码：

Static (静态高斯字典)：根据语音信号统计特性，大量研究表明，人类语音的幅值分布近似服从高斯分布，直接利用先验知识，假设所有语音都符合高斯分布，这样在传输过程中不需要传递字典，通信开销小，抗误码强

Huffman 算法的不确定性：当两个符号概率完全相等时，Huffman 树的构建取决于排序算法的稳定性或具体的实现细节。如果发送端和接收端计算精度有微小差异，两端生成的字典编码可能会互换。这将导致解码数据不仅错误，因此需要因为 linspace 破坏这种对称性

```
x_axis = linspace(-3, 3, 256);
pdf_sim = exp(-x_axis.^2) + linspace(0, 1e-10, 256);
p = pdf_sim / sum(pdf_sim);
dict = huffmandict(symbols, p);
```

```
   2. 模拟信号: 启动 (Go)
   3. 模拟信号: 停止 (Stop)
   4. 模拟信号: 波动 (Wave)
>>> 请输入选项 (1/2/3/4): 2
   [Huffman编码] 模式: Static (通用高斯模型)
   [信源编码] 原始: 64000 bits -> Huffman: 49376 bits (压缩率: 22.85%)
```

Adaptive (自适应字典)：基于后验概率统计了当前这一秒钟语音的真实分布，完全根据当前数据的特性构建最优树，可以得到更高的压缩率，但是相应的这个统计表也需要发送给接收端才可以解码出原码

在统计频率的过程中，由于语音信号绝大部分还是静音信号，为了节省发送开销，选择计算相对频次，这个相对频次拉伸到 0 到 255（8bit 信息），然后再计算频率

```
% 统计频次
raw_counts = histcounts(q_indices, -0.5:255.5);
raw_counts = raw_counts + 1;
% 归一化并生成字典头
max_c = max(raw_counts);
norm_counts = round(raw_counts / max_c * 255);
norm_counts(norm_counts == 0) = 1;
p = norm_counts / sum(norm_counts);
dict = huffmandict(symbols, p);
% 生成字典头
dict_header_mat = decimalToBinaryVector(norm_counts, 8);
dict_header_bits = reshape(dict_header_mat', [], 1);
```

```
    2. 模拟信号: 启动 (Go)
    3. 模拟信号: 停止 (Stop)
    4. 模拟信号: 波动 (Wave)
>>> 请输入选项 (1/2/3/4): 2
    [Huffman编码] 模式: Adaptive (统计当前信号)
    [信源编码] 原始: 64000 bits -> Huffman: 34794 bits (压缩率: 45.63%)
```

从不同 Huffman 的压缩比可以看出，自适应的 Huffman 由于统计了实际的后验分布，所以压缩率会更高

（3）构建数据包：相当于构建了一个通信协议

结构: [模式] + [数据长度] + [dict] + [payload]

模式标志，位于数据包的最开头，决定解码策略；数据长度为定界符，分配了 16 个比特来存储长度值，这是编码的有效范围；dict，如果选择了自适应的 Huffman 编码，则需要将字典传入；payload 则为发送端编码后的序列

```
% 模式标志
header_mode = mode_bit;
% 有效数据长度
len_val = length(huff_bits);
header_len = decimalToBinaryVector(len_val, 16)';
% 拼接
raw_bits = [header_mode; header_len; dict_header_bits; huff_bits];
```

3．信道编码与帧同步

香农第二定理告诉我们，如果信息传输速率 R 小于信道容量 C，则理论上存在一种信道编码方法，使得误码率 Pe 可以任意接近于 0，Hamming 码就是一种尝试逼近这一极限的手段。虽然 Hamming(7,4) 不能做到零误码，但它在牺牲了 3/7 的带宽冗余后，换取了系统

在低信噪比下生存的能力

Hamming(7,4)分组码，通过增加冗余，将数据映射到一个更高维的几何空间中，使得数据点之间保持足够的距离。[3]码长为 7，信息位为 4，能且仅能纠错 1 位错码

发送端：生成矩阵 $G=\begin{bmatrix}1&1&0&1&0&0\\1&0&1&0&1&0\\0&1&1&0&0&1\\1&1&1&0&0&0\end{bmatrix}_{4\times7}$，设原始信息向量为 $\mathbf{m}=[d_1\quad d_2\quad d_3\quad d_4]$，然后 $c=m\times G\ mod2$，得到 7 位额度 Hamming 码，其中 $c_1=d_1+d_2+d_4$，$c_2=d_1+d_3+d_4$，$c_3=d_2+d_3+d_4$，前三位为校验位，$c_4=d_1$，$c_5=d_2$，$c_6=d_3$，$c_7=d_4$，后四位为原始数据

接收端：构造矩阵 $H=\begin{bmatrix}1&0&0&1&1&0&1\\0&1&0&1&0&1&1\\0&0&1&0&1&1&1\end{bmatrix}_{3\times7}$，其中 G 和 H 满足正交，即 $G\times H^T=0\ mod2$，接受到的信号定义为 r，其中可能包含错误向量为 e（只有一项为 1，此项为错误项），计算伴随式为 $S=r\times H^T=(c+e)\times H^T=0+e\times H^T=e\times H^T=$ H 第 j 列，可以看出结果此时只与错误 e 有关，为 H 的第 j 列进行编码，可以根据唯一的编码确定原来 e 对应的错误项的位置，是一个标准的线性分组纠错过程

```
n_pads = mod(4 - mod(length(raw_bits), 4), 4);
if n_pads == 4, n_pads = 0; end
raw_bits_padded = [raw_bits; zeros(n_pads, 1)];
msg_blocks = reshape(raw_bits_padded, 4, [])';
% 生成矩阵 G
G = [1 1 0 1 0 0 0; 1 0 1 0 1 0 0; 0 1 1 0 0 1 0; 1 1 1 0 0 0 1];
code_blocks = mod(msg_blocks * G, 2);
payload_stream = reshape(code_blocks', [], 1);
```

```
>>> 请输入选项 (1/2/3/4): 2
    [Huffman编码] 模式: Adaptive (统计当前信号)
    [信源编码] 原始: 64000 bits -> Huffman: 34794 bits (压缩率: 45.63%)
    [帧结构] 添加同步头(13bit) + 数据(64505bit)
    [Huffman解码] 模式: Adaptive
    >>> 信道状态(未纠错): 误码率 0.0070 | 汉明码成功纠正了 440 个错误
    >>> 最终误码率:  0.0007 (纠错后残留错误)
```

在信道信噪比较低的情况在，引入 Hamming 纠错码可以有效降低误码率，但是也可以看出，Huffman 编码后只有 34794bits，但是 Hamming 纠错码的引入使得数据量变为 64505bit，引入了较多的数据冗余

帧同步技术: 在数据包头部插入 13 位 Barker 码 ([1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1])。接收端利用滑动互相关检测峰值来确定数据帧起始位置，Barker 码具有尖锐的自相关特性，能有效防止噪声引起的同步滑移，在帧头前增加一点“保护位”，让滤波器进入稳态后再发送关键的 Barker 码

$$R_{xy}(\tau) = \sum_{n} x(n)y(n-\tau)$$

```
barker_code = [1; 1; 1; 1; 1; 0; 0; 1; 1; 0; 1; 0; 1];
preamble = zeros(32, 1);
encoded_stream = [preamble; barker_code; payload_stream];
% 最终输出
encoded_stream = [barker_code; payload_stream];
```

4．BPSK 调制与相干解调

二元 PSK(2PSK,也称 BPSK)，其中数据“0”和“1”控制相位“0”和“π”的切换。这是一种对映信号，与 M=2 的对称双极性幅度调制一样[4]

$$s(t) = \sum_{k=-\infty}^{\infty} \cos(2\pi f_c t + \theta + \phi_k) g_T(t - kT) = x(t)\cos(2\pi f_c t + \theta) - y(t)\sin(2\pi f_c t + \theta)$$

对于 A 类 BPSK，已调信号可以写为：

$$s(t) = \sum_{k=-\infty}^{\infty} \cos(2\pi f_c t + \varphi_k) g_T(t - kT),\ \varphi_k = \{0, \pi\}$$

$$s(t) = \sum_{k=-\infty}^{\infty} I_k G_T(t - kT)\cos[2\pi f_c(t - kT)]$$

在一个符号间隔中考虑码元信号：

$$s_0(t) = -g_T(t)\cos(2\pi f_c t)，\text{发送“0”}$$

$$s_1(t) = g_T(t)\cos(2\pi f_c t)，\text{发送“1”}$$

（1）定义 BPSK 调制参数：

```
%% 1. 参数
Rb = 10000; Fc = 40000; Fs = 160000; sps = Fs / Rb;
```

（2）进行调制，先映射为双极性码元实现相位变化，由于序列可能较短，上采样使得每个采样点扩展 Fs/Rb 次，然后基带成型为矩形波，$m(t) = \sum_n a_n \cdot g(t - nT_b)$，然后乘以载波，将

频谱搬移到载波附近，$s(t) = m(t) \cdot \cos(2\pi F_c t)$

```
%% 2. 调制
bipolar_msg = 2*tx_bits - 1;
tx_baseband = repelem(bipolar_msg, sps);
t = (0:length(tx_baseband)-1)' / Fs;
carrier = cos(2*pi*Fc*t);
tx_passband = tx_baseband .* carrier;
```

（3）叠加高斯噪声，调制信号为单位幅度的余弦波信号，可以算出其平均功率为 0.5，然后根据既定的信道 SNR 算出噪声，叠加 AGWN 加性高斯白噪声

```
%% 3. AWGN 信道
sig_power = 0.5;
noise_power = sig_power / (10^(snr_db/10));
noise = sqrt(noise_power) * randn(size(tx_passband));
rx_passband = tx_passband + noise;
```

（4）相干解调：乘载波，$y(t) = r(t) \times 2\cos(2\pi F_c t) = m(t) + m(t)\cos(2 \cdot 2\pi F_c t)$，得到了基带分量和二倍频分量，然后乘低通滤波器滤除高频分量，抽样发生在波形最稳定的时刻，这里为符号周期的中间，双极性码的判决门限设为 0，大于 0 则判为 1，小于 0 则判为 0

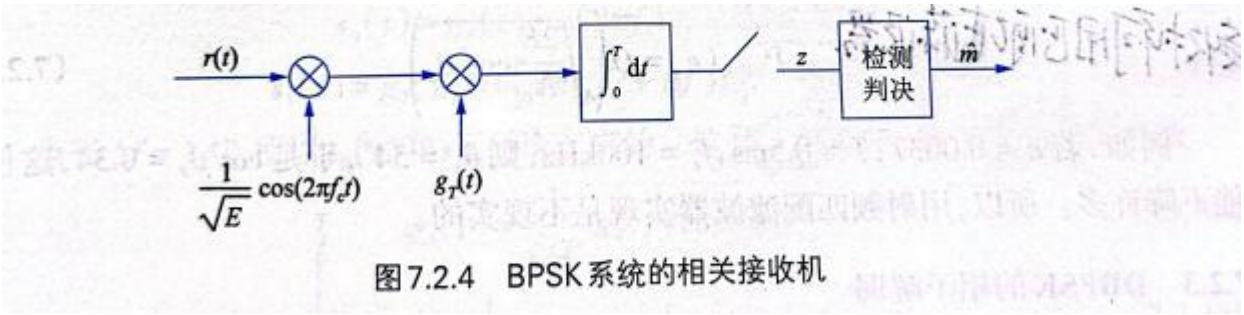
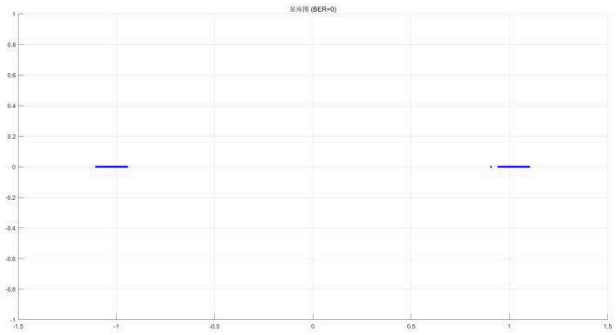
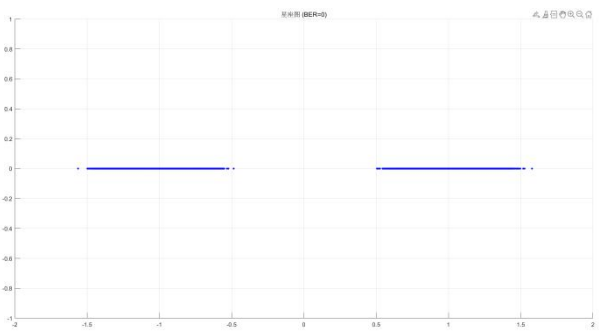
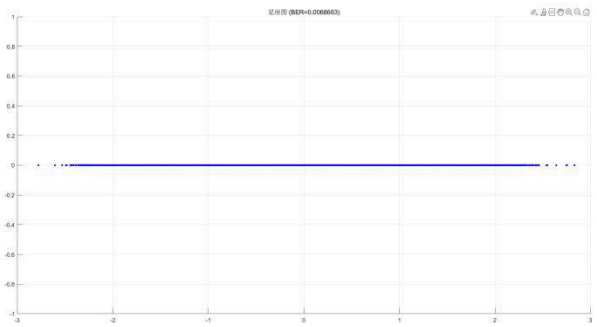
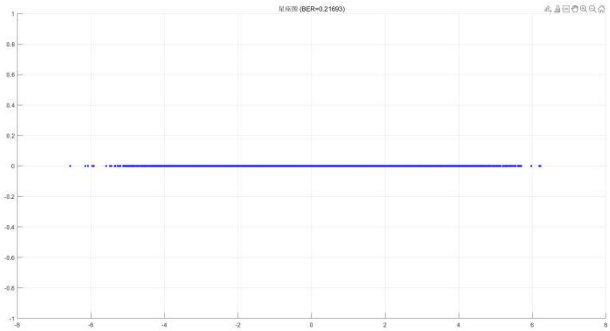


图 7.2.4　BPSK 系统的相关接收机

```
%% 4. 相干解调
rx_mixed = rx_passband .* carrier * 2;
[b, a] = butter(5, (1.5 * Rb) / (Fs/2));
rx_filtered = filtfilt(b, a, rx_mixed);% 零相位滤波器
%% 5. 抽样判决
sample_indices = round(sps/2 : sps : length(rx_filtered));
rx_sampled = rx_filtered(sample_indices);
rx_bits = zeros(size(tx_bits));
rx_bits(rx_sampled > 0) = 1;
```

下面分别为 SNR=30dB、10dB、0dB、-10dB 的星座图

SNR = 30dB 时，星座点几乎重合为两个清晰的点，欧几里得距离非常清晰，误码率 (BER) 理论上趋近于 0；SNR = 10dB 时，星座点开始扩散成“团簇状”，但两个团簇之间仍有明显的空白隔离带，噪声开始引入随机相位和幅度的抖动，但抖动幅度未超过判决门限；SNR=0dB 时，团簇显著扩大，边缘开始接近中心判决线，部分采样点可能会因为瞬时噪声过大导致误判；SNR=-10dB 时，左右两个团簇彻底混合在一起，无法区分中心，大部分采样点都可能越过判决门限，带来较高的误码率 BER

5．帧同步与信道解码

帧同步的目的是在连续的比特流中找到数据包的起始位置，使用互相关检测，设接收到的比特序列为 x[n]，Barker-13 同步头序列为 h[n]，则互相关函数为：$R_{xh}[k] = \sum_{i=0}^{L-1} x[n+i] \cdot h[i]$，Barker 码具有理想的自相关特性，当 x[n]与 h[n]完全对齐时，相关值达到最大值 13，由此可以从连续 bit stream 中找到帧头起始位

由于 Adaptive 在数据包非常长，而 Barker 码只有 13 位，统计上大概会出现多次“伪帧头”在信号最开始的边缘处，滤波器可能会产生瞬态效应，导致真实帧头的某个 bit 幅度不够完美，相关性得分可能只有 12 或 12.5。 而在数据包内部的那些“伪帧头”，因为周围数据平稳，相关性得分可能是完美的 13，可能会导致定位错误

```
[correlation, lags] = xcorr(rx_bipolar, head_bipolar);
% 找到相关性最大位置
corr_threshold = 10;
```

```
candidate_indices = find(correlation > corr_threshold);
if isempty(candidate_indices)
warning('帧同步失败：未找到有效帧头。');
recovered_signal = zeros(100,1); num_errs = 0; final_bits=[]; return;
end
% 在候选点中找到 lag 最接近 0 的
valid_candidates = candidate_indices(lags(candidate_indices) >= 0);
if isempty(valid_candidates)
[~, best_idx_in_candidates] = min(abs(lags(candidate_indices)));
max_idx = candidate_indices(best_idx_in_candidates);
else
[~, best_idx_in_candidates] = min(lags(valid_candidates));
max_idx = valid_candidates(best_idx_in_candidates);
end
peak_lag = lags(max_idx);
% 索引
start_idx = peak_lag + 1;
payload_start_idx = start_idx + length(header_template);
```

信道解码目标是纠正传输过程中产生的随机误码

构造校验矩阵$H=\begin{bmatrix}1&0&0&1&1&0&1\\0&1&0&1&0&1&1\\0&0&1&0&1&1&1\end{bmatrix}$，计算伴随式 $S=r\cdot H^T=(c+e)\cdot H^T=0+eH^T=eH^T$，若 S=0，则此处无误码；若 S≠0，则 S 的值对应矩阵 H 中某一列的二进制表示，该列的位置即为错误位的位置，Hamming(7,4)码后四位为原始信息，根据之前得到的错误位置反转对应的原始信息

```
% 生成校验矩阵 H
H = [1 0 0 1 1 0 1; 0 1 0 1 0 1 1; 0 0 1 0 1 1 1];
rx_blocks = reshape(rx_payload_aligned, 7, n_blocks)';
% 计算伴随式 & 纠错
S = mod(rx_blocks * H', 2);
syndrome_idx = S * [4; 2; 1]; % 将二进制转为十进制
col_vals = [4 2 1 6 5 3 7]; % 对应 H 矩阵各列
rx_corrected = rx_blocks;
num_errs = 0;
for i = 1:n_blocks
syn = syndrome_idx(i);
if syn ~= 0
err_pos = find(col_vals == syn);
if ~isempty(err_pos)
rx_corrected(i, err_pos) = ~rx_corrected(i, err_pos);
num_errs = num_errs + 1;
end
```

```
end
end
```

6. 信源解码

（1）协议解析：由于传入的为数据流[Mode] + [Length] + [Dict] + [Payload]，因此需要分别读取和解析

①读取模式位

```
current_idx = 1;
if length(total_stream) < 17
warning('数据包太短，无法解析头部。');
recovered_signal = zeros(100,1); return;
end
mode_bit = total_stream(current_idx);
current_idx = current_idx + 1;
```

②读取数据长度

```
len_bits = total_stream(current_idx : current_idx+15);
data_len = bi2de(len_bits', 'left-msb');
current_idx = current_idx + 16;
symbols = 0:255;
```

③解析 Huffman 字典

如果是 Static，则发送端并没有发送来 dict 字典，因此需要本地构建一个符合语音信号的，和发送端相同的字典

```
x_axis = linspace(-3, 3, 256);
pdf_sim = exp(-x_axis.^2) + linspace(0, 1e-10, 256);
p = pdf_sim / sum(pdf_sim);
dict = huffmandict(symbols, p);
```

如果是 Adaptive，则需要读取数据头，然后将其反归一化为对应的频率，然后重建 dict

```
dict_len = 256 * 8;
dict_bits = total_stream(current_idx : current_idx + dict_len - 1);
current_idx = current_idx + dict_len;
dict_mat = reshape(dict_bits, 8, 256)';
rx_counts = bi2de(dict_mat, 'left-msb');
rx_counts(rx_counts==0) = 1; % 防止 0 概率
p = rx_counts / sum(rx_counts);
dict = huffmandict(symbols, p);
```

④提取 Huffman 编码结果：

```
huff_bits = total_stream(current_idx : current_idx + data_len - 1);
```

（2）Huffman 解码：利用重建的二叉树，对 Payload 比特流进行遍历。从根节点出发，读取 0 向左，读取 1 向右，直至到达叶子节点，输出对应的量化索引 q

```
dsig_indices = huffmandeco(huff_bits, dict);
```

（3）PCM 逆量化：将离散的整数索引 q 映射回连续的电压幅值$\hat{x} = q \cdot \Delta + x_{\min} + \frac{\Delta}{2}$

```
min_val = -1; max_val = 1; L = 256;
delta = (max_val - min_val) / L;
% 映射回幅度值
recovered_signal = dsig_indices * delta + min_val + delta/2;
```

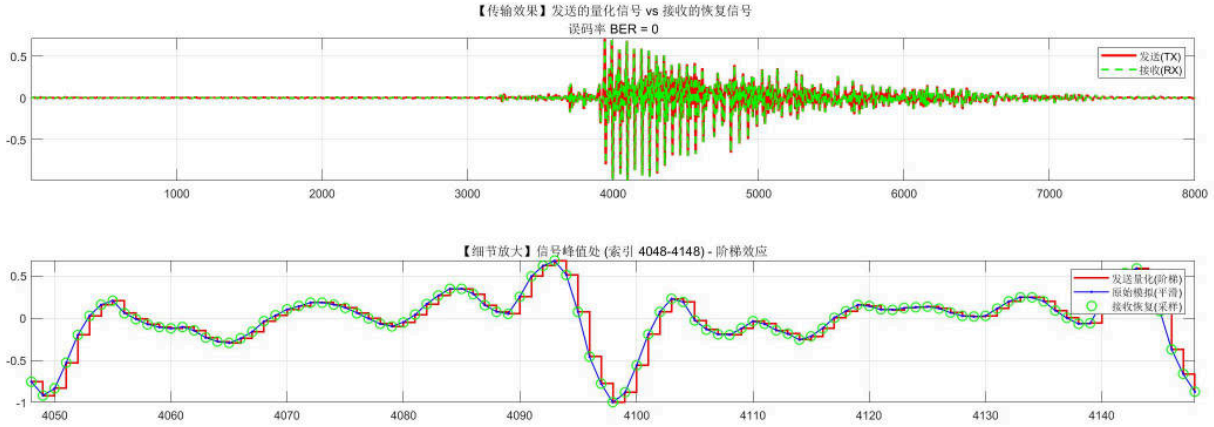
以下为不同信噪比 SNR=30dB、10dB、0dB、-10dB 的解码结果：

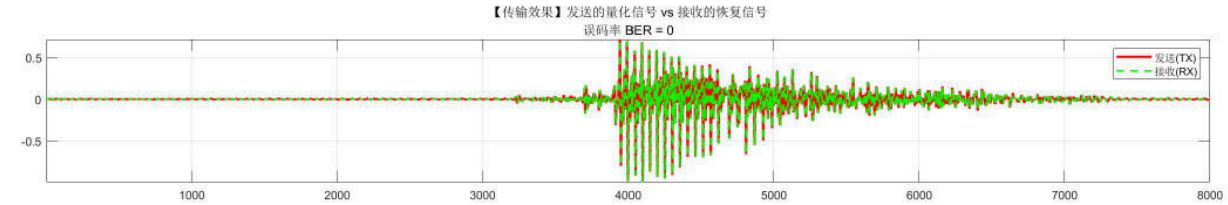
（1）SNR=30、10dB，波形：发送波形（红线）与接收波形（绿线）完美重合，系统在这些环境下具有极高的鲁棒性，量化噪声（PCM 带来的阶梯效应）是唯一的失真来源，传输过程是无损的

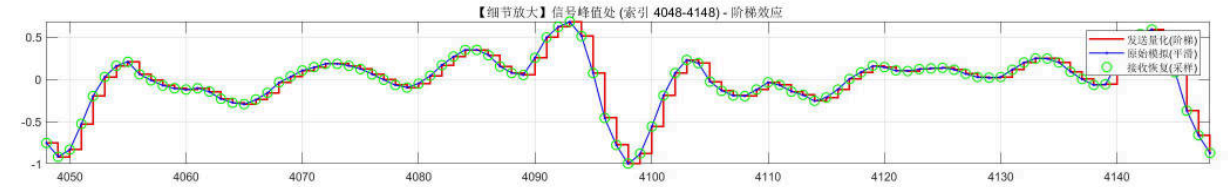
（2）SNR=0dB 时，绿线（接收）偶尔会跳开，不跟随红线，汉明码纠错失败的残留误码，当一个 7 位码块中出现 2 bit 以上错误时，汉明码不仅修不好，可能还会修错，导致这一段 PCM 数据解出来是错的

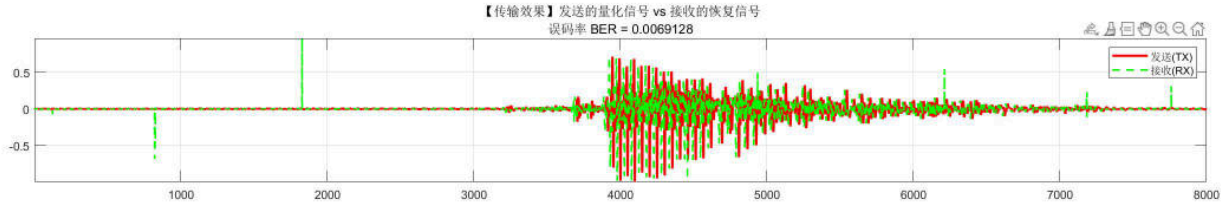
（3）SNR=-10dB 时，物理层有 22% 的位是错的，错误极大概率击中了协议头中的数据长度字段或者是 Huffman 字典字段，一旦长度字段读错，接收端截取的数据包长度就不对；一旦字典头读错，Huffman 树就建错了，后续的所有数据虽然收到了，但全是按照错误的规则解码的。解出来的比特流就变成了纯随机数，所以误码率变成了 0.5

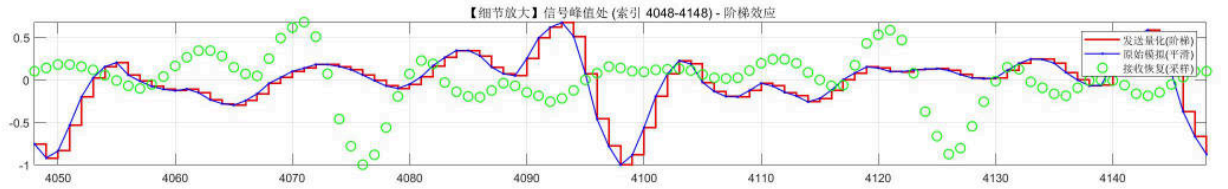
警告的来源是：一方面可能是帧同步失败，产生了同步漂移，真正的帧头在第 1 个点，但被噪声淹没了，相关性很低，使得序列起始点判断错误；另一方面是数据长度字段误码，解码器会把填充零当成数据读进去，或者把有效数据截断，使得发送和接收的 bit 数不同

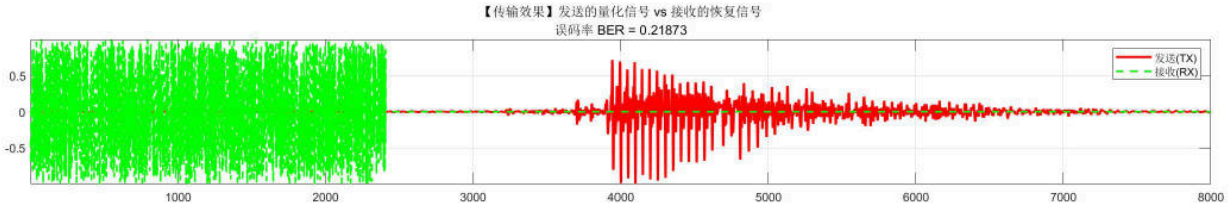


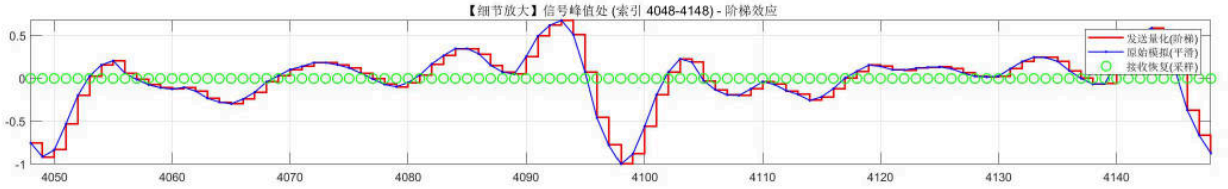













```
命令行窗口
    3. 模拟信号: 停止 (Stop)
    4. 模拟信号: 波动 (Wave)
>>> 请输入选项 (1/2/3/4): 2
    [Huffman编码] 模式: Adaptive (统计当前信号)
    [信源编码] 原始: 64000 bits -> Huffman: 34794 bits (压缩率: 45.63%)
    [帧结构] 添加同步头(13bit) + 数据(64505bit)
    [Huffman解码] 模式: Adaptive
    [警告] 长度不匹配: 发送 36860 bits vs 接收 28764 bits (已对齐计算)
    >>> 信道状态(未纠错): 误码率 0.2190 | 汉明码成功纠正了 6293 个错误
    >>> 最终误码率:  0.5013 (纠错后残留错误)
```

7. 子系统总结

经过上述通信系统处理后，模拟了采集不同的语音指示信号，进行适当压缩；然后通过

噪声信道实现远程传输的通信系统

（1）信源编码的高效性：实验对比了两种 Huffman 编码策略，结果显示自适应模式 相比静态模式具有更高的压缩效率。验证了基于后验概率统计的编码方案能更好地去除语音信号中的数据冗余，但代价是增加了传输字典头的开销。

（2）PCM 量化保真度：在 8-bit 量化深度下，量化信噪比很高，重构信号与原始信号的时域波形高度重合，证明了均匀量化引入的量化噪声极低

（3）本通信系统的可靠工作阈值约为 0dB，而在极低信噪比下，协议头部的保护机制（如更强的纠错码或交织技术）是系统性能的瓶颈

五、系统 II:计算语音识别子系统

系统基于经典声学特征与 KNN 分类器，这在计算资源受限的嵌入式端尤为适用，对于指令少、环境相对简单的任务来说，具有计算量小、响应快、可解释性强的优点

1．特征工程

首先进行预处理，去除直流分量和静音片段，只保留有效语音。去直流：麦克风录音时可能会叠加一个恒定的直流电压偏移。如果不去除，这会被视为 0Hz 的巨大能量，严重干扰频谱质心的计算$x_{new}[n] = x_{old}[n] - \frac{1}{N}\sum_{i=1}^{N} x_{old}[i]$；端点检测：录音开头和结尾通常包含背景噪声。为了防止把噪声算进特征里，需要截取有效语音段[5]

提取了语音命令信号三个具有物理意义的特征向量：[6]

（1）过零率：反映信号频率特性的时域指标，信号穿过零轴的频繁程度，"Wave"：低频正弦波，变化缓慢，ZCR 较低；"Stop"：包含清辅音 /s/，是高频噪声，ZCR 较高

$$ZCR = \frac{1}{2N}\sum_{n=1}^{N} |sgn[x(n)] - sgn[x(n-1)]|$$

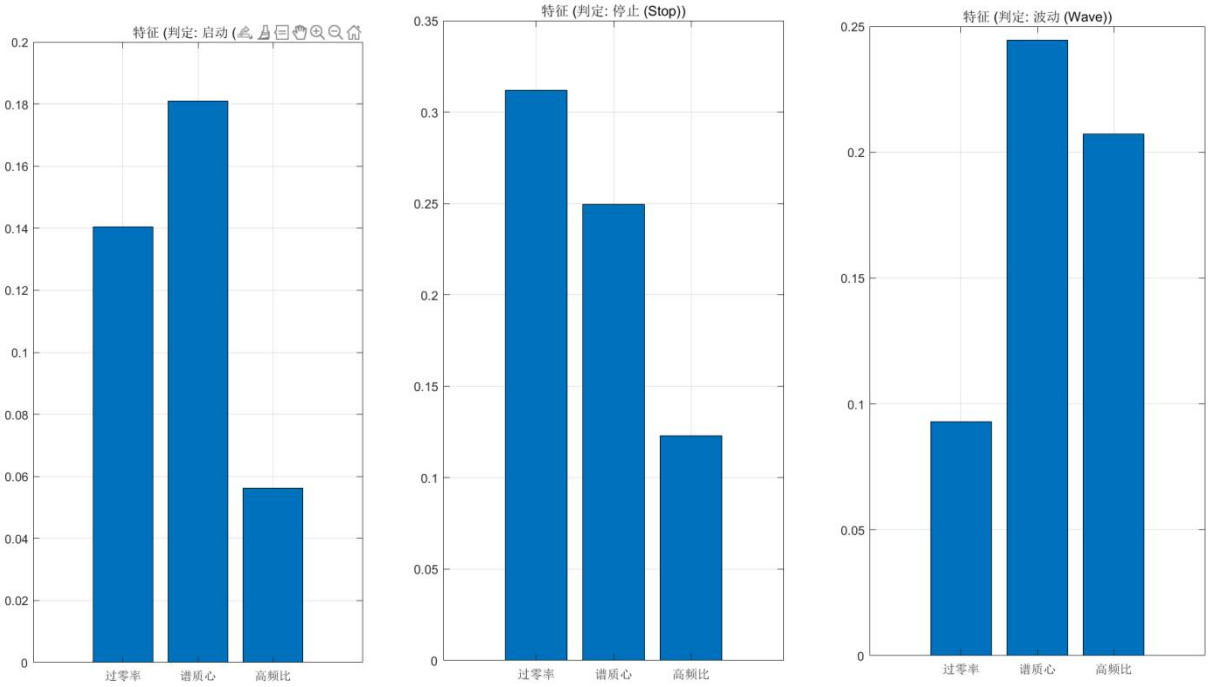
（2）谱质心：频域能量的“重心”，区分低频元音与高频辅音，反映了声音的“亮度”，"Go"：能量集中在低频，质心较低；"Stop"：能量分布较宽且有高频分量，质心较高

$$C = \frac{\sum_k f(k)|X(k)|}{\sum_k |X(k)|}$$

（3）高频能量占比："Stop"：结尾的 /p/ 或开头的 /s/ 是爆破/摩擦音，在 2000Hz 以上有

显著能量；"Go"：能量几乎全在 1000Hz 以下

$$R_{HF} = \frac{\int_{2000}^{f_s/2} |X(f)| df}{\int_{0}^{f_s/2} |X(f)| df} \approx \frac{\sum_{k:f_k>2000} P[k]}{\sum_{all\ k} P[k]}$$



2．KNN 分类器

得到特征向量[ZCR，Centroid，Ratio] 后，使用 K-最近邻算法进行判决[7]

（1）模型训练：系统存储了这 3 个指令的参考特征点（本系统中每个指令录了 3 次，空间里有 9 个参考点）

（2）预测：当一个新的录音进来，提取出它的特征点 P

（3）度量：计算 P 与空间中所有 9 个参考点的欧几里得距离

（4）决策：找到距离最近的一个点，那个点属于哪个指令，就把新录音判定为哪个指令

$$\mathbf{x} = (x_1, x_2, x_3),\ \mathbf{y}^{(i)} = \left(y_1^{(i)}, y_2^{(i)}, y_3^{(i)}\right)$$

$$D_i = \sqrt{(x_1 - y_1^{(i)})^2 + (x_2 - y_2^{(i)})^2 + (x_3 - y_3^{(i)})^2},\ \widehat{C} = \text{Label}\left(arg\min_i \quad D_i\right)$$

3．子系统总结

该子系统证明了在计算资源受限的场景下，通过精细的物理特征提取（时域+频域）可以替代复杂的深度学习模型，高效地解决特定场景下的意图识别问题

六、系统 III:现代控制系统设计

1．物理对象建模

系统设定为一个质量-弹簧-阻尼系统

物理方程角度看，系统的动力学方程为：$m\ddot{x}(t)+c\dot{x}(t)+kx(t)=u(t)$，m 为质量，k 为弹簧劲度系数，c 为阻尼系数，u 为输入力（控制量），x 为位移

状态空间方程看，设 x1 为位移，x2 为速度，则系统方程为：$\begin{cases}\dot{x}_1=x_2\\ \dot{x}_2=-\frac{k}{m}x_1-\frac{c}{m}x_2+\frac{1}{m}u\end{cases}$，写为矩阵的形式后，$\dot{x}=\begin{bmatrix}0 & 1\\ -k/m & -c/m\end{bmatrix}x+\begin{bmatrix}0\\ 1/\mathrm{m}\end{bmatrix}u=Ax+B$，$y=\begin{bmatrix}1\\ 0\end{bmatrix}x$，由于要设计一个处于未知状态的系统，因此设定传感器只能测量位置，无法直接测量速度[8]

能控性和能观性分析：

能控性：判别矩阵$Q_c=[B \quad AB]$满秩，系统状态可任意配置；

能观性：判别矩阵$Q_o=[C^T \quad A^TC^T]^T$满秩，表明尽管只能测量位移 y，但可以通过输出推断出速度 x2

```
m = 1; % 质量
k = 15; % 弹簧系数
c = 2.5; % 阻尼系数
% 状态矩阵
A = [0 1;
-k/m -c/m];
B = [0;
1/m];
C = [1 0]; % 只能观测到位置(x1)，看不见速度(x2)
D = 0;
if rank(ctrb(A,B)) < length(A)
error('系统不可控 (Uncontrollable)');
end
if rank(obsv(A,C)) < length(A)
error('系统不可观 (Unobservable) ');
end
```

2．状态观测器设计

要实现精准控制，控制器需要知道当前的速度 x2，但只有一个位置传感器。因此设计一个龙伯格观测器，用来估计系统的内部状态[9]

观测器方程：$\dot{\hat{x}}=A\hat{x}+Bu+L(y-C\hat{x})$，其中 xhead 是估计的状态，y-Cxhead 是估计误

差，L 是观测器的增益矩阵；采用极点配置的方法计算 L 矩阵，配置目标极点为[-10,-12]，矩阵 A - LC 的特征值被配置为-10 和-12，保证估计误差$e(t) = x(t) - \hat{x}(t)$快速收敛

```
desired_poles_ctrl = [-2, -3];
K = place(A, B, desired_poles_ctrl);
```

有了估计出的完整状态 xhead，可以使用状态反馈来镇定系统，配置目标极点为[-2,-3]，矩阵 A - BK 的特征值被配置为-2 和-3

```
desired_poles_obs = [-10, -12];
L = place(A', C', desired_poles_obs)';
```

3．控制律设计

单纯的$u = -Kx$会把系统状态驱动到 0，但目前是需要系统根据指令前往不同的位置，所以引入前馈增益，最终的控制律为$u(t) = -K\hat{x}(t) + \bar{N}r(t)$，$K\hat{x}(t)$反馈项，负责稳定系统，消除误差，$\bar{N}r(t)$前馈项，负责让系统能够到达指定位置

为了保证稳态误差为零，使用了逆矩阵法解算平衡点，当输出 y=1 且状态导数为 0 时，系统需要的状态 Nx 和输入 Nu 是多少。最终算出的$\bar{N} = N_u + KN_x$补偿了弹簧力，让质量块能停在 x=1 的位置

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} N_x \\ N_u \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

```
s = size(A,1);
Z = [zeros([1,s]) 1];
N = inv([A,B;C,D])*Z';
Nx = N(1:s); % 稳态状态
Nu = N(1+s); % 稳态输入
N_bar = Nu + K*Nx;
```

4．仿真循环

首先进行物理系统演化，对真实情况进行模拟，根据牛顿定律，力 u 产生加速度，改变速度，进而改变位置

```
dx_real = A * x_real + B * u;负责计算变化率
x_real = x_real + dx_real * dt; 负责更新位置状态
```

然后根据真实情况的模拟，对观测器进行更新，相比真实的物理系统多了一项矫正项，整个过程为：先预测输出位置，然后计算真实传感器得到的位置和预测位置的残差，根据残差修改估计的导数，然后将导数代入更新估计的状态

```
y_est = C * x_hat; [10]
dx_hat = A * x_hat + B * u + L * (y - y_est);
x_hat = x_hat + dx_hat * dt;
```

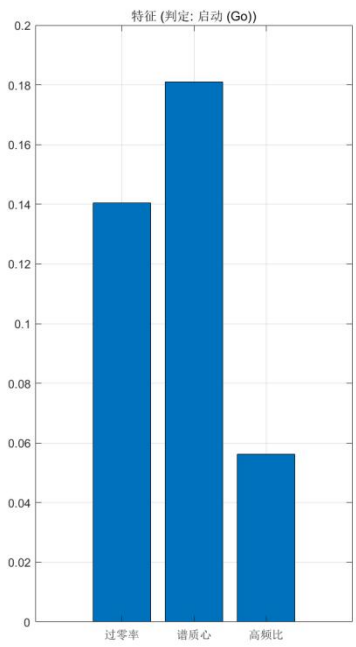
以下分别为输入命令为 Go、Stop、Wave 后得到系统响应和控制效果

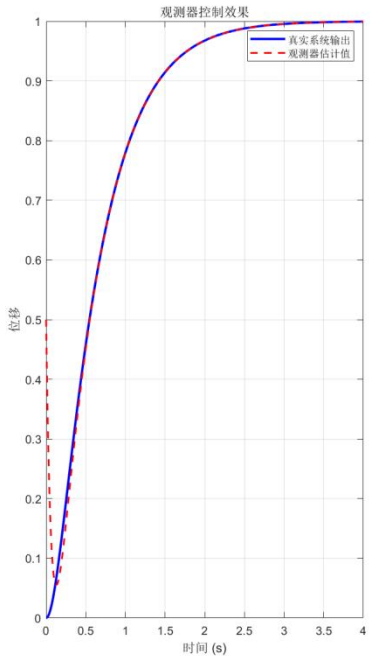
（1）Go 指令：阶跃响应，系统从 0 开始上升，大约在 1.0 秒 左右达到目标值，且没有明显的超调，最终稳稳地停在了 y=1 处，系统实现了无静差跟踪

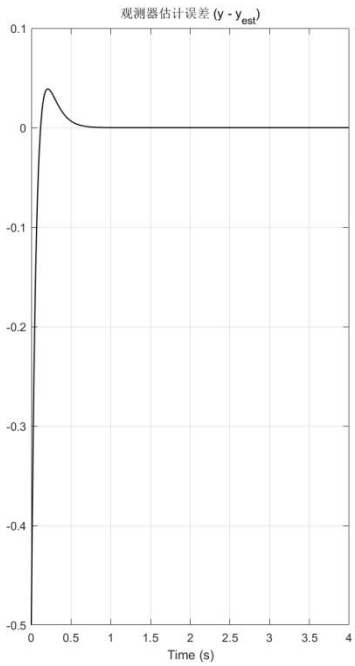
（2）Stop 指令：归零，系统受到初始扰动有一个小的波动，随后迅速被拉回并锁定在 y=0，表现出了良好的抗扰动能力和稳态精度

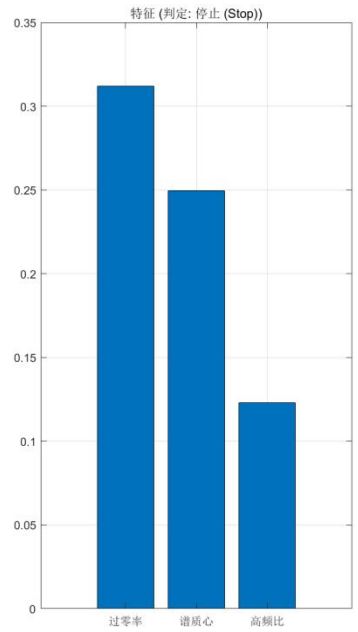
（3）Wave 指令：系统输出完美地复现了正弦波形，幅值保持在 0.5，且相位滞后极小，控制系统不仅能做定点控制，也能做动态轨迹跟踪

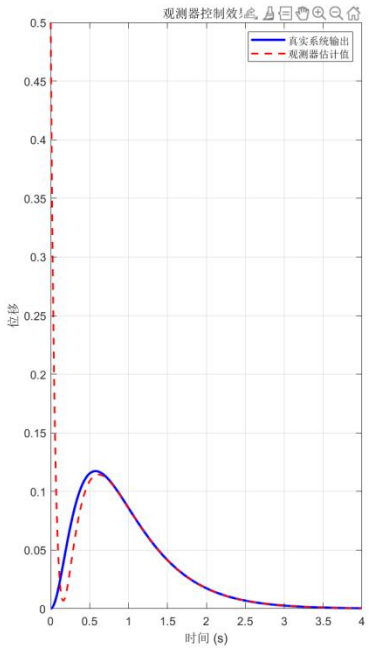
（4）状态观测器在 0.5 秒内实现了对真实状态的收敛，成功解决了速度未知的系统控

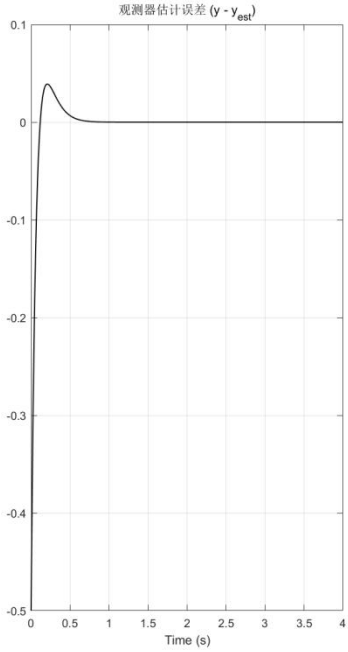


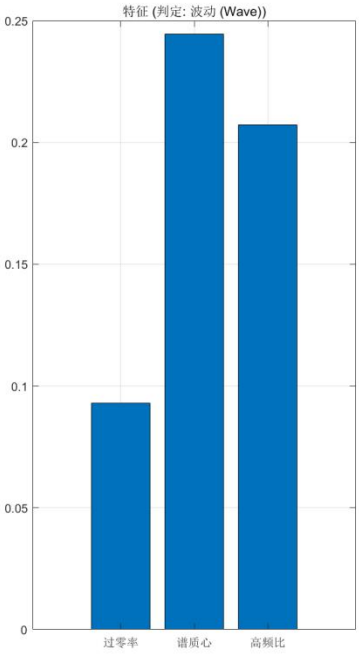


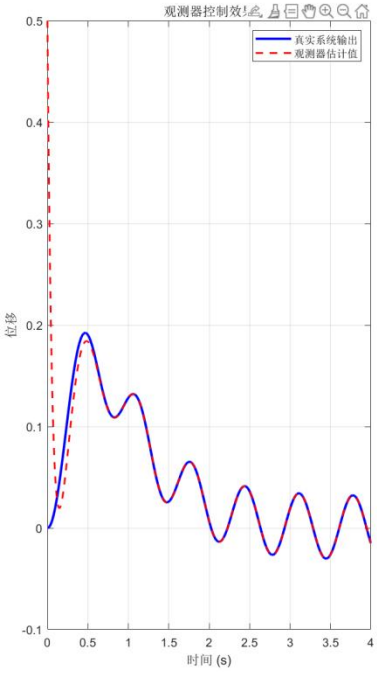


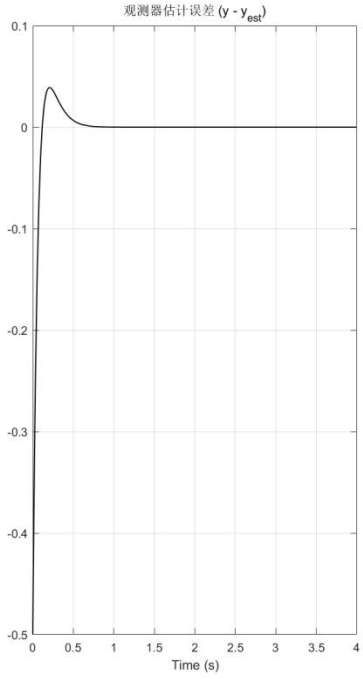












5．子系统总结

（1）状态观测的收敛性：实验结果显示，龙伯格观测器的估计误差 e(t)在 0.5 秒内迅速收敛至 0。阶跃响应：系统实现了无静差跟踪，证明前馈增益矩阵 N 准确计算并抵消了弹簧的回复力；抗扰动：系统在非零初始状态下能迅速镇定回零

（2）该子系统成功构建了一个“特征识别-参考输入-状态重构-闭环反馈”的完整控制链路，验证了极点配置与观测器分离原理在远程物理控制中的工程可行性

七、总结与心得

本次“信息控制与计算”课程设计让我受益匪浅。通过构建这个从语音采集、数字传输到远程物理控制的完整闭环系统，我深刻体会到了信息流、计算决策与物理控制三者之间的紧密耦合关系。

在信息与计算层面，实验让我走出了理论课本。我验证了 Huffman 编码在信源压缩中的高效性，更通过模拟恶劣信道，认识到 Hamming 纠错码与 Barker 帧同步对于保障“信息完整性”的决定性作用。在计算环节，简单的 KNN 算法配合精细的物理特征提取即可实现高精度识别，这让我理解了“计算”在特征降维与意图理解中的核心价值。

在控制层面，现代控制理论的应用是本次实验的升华。针对二阶系统速度不可测的工程痛点，龙伯格观测器的设计巧妙地利用输出误差重构了系统状态，配合极点配置实现了精准的闭环控制。这种“利用数学模型填补物理缺陷”的思想让我印象深刻。

综上所述，本实验不仅锻炼了我的 MATLAB 建模与仿真能力，更重要的是培养了我的全链路工程视角。我意识到系统不再是孤立的模块，前端的通信误码会直接导致后端的控制崩溃。这种跨学科的理论融合与实践，为我未来解决复杂工程问题打下了坚实的基础。

八、参考文献

[1] 门爱东, 苏菲, 王雷,等. 数字信号处理[M]. 科学出版社, 2005.

[2] 仇佩亮, 张朝阳, 谢磊, 余官定,等. 信息论与编码.第 2 版[M]. 高等教育出版社, 2011.

[3] Proakis J G, Salehi M. Digital Communications (5th Edition) [M]. New York: McGraw-Hill, 2008.

[4] 樊昌信, 曹丽娜. 通信原理 (第 7 版) [M]. 北京: 国防工业出版社, 2012.

[5] Rabiner L R, Schafer R W. Theory and Applications of Digital Speech Processing [M]. Pearson, 2010.

[6] 韩纪庆, 张磊, 郑铁然. 语音信号处理 (第 3 版) [M]. 北京: 清华大学出版社, 2019.

[7] 李航. 统计学习方法 (第 2 版) [M]. 北京: 清华大学出版社, 2019.

[8] 刘豹, 唐万生. 现代控制理论 (第 3 版) [M]. 北京: 机械工业出版社, 2011.

[9] Ogata K. Modern Control Engineering (5th Edition) [M]. Prentice Hall, 2010.

[10] 邵玉斌. MATLAB/Simulink 通信系统建模与仿真实例分析 [M]. 北京: 清华大学出版社, 2009.